

INSTITUTO MAUÁ DE TECNOLOGIA



Lógica e Pensamento Computacional

Revisão – P1

OBJETIVOS

- Revisar os conceitos das aulas anteriores

Algoritmo: Sequência finita, ordenada e não ambígua de passos (regras/instruções) para resolver um problema ou executar uma tarefa.

- Um algoritmo pode ser representado de diversas formas, onde destacaremos duas delas: Fluxograma e Código em linguagem computacional

Fluxograma: “Representação gráfica da definição, análise e solução de um problema na qual são empregados símbolos geométricos e notações simbólicas”



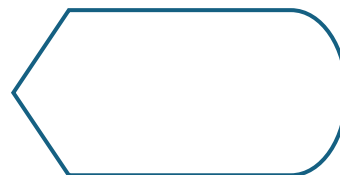
Terminador



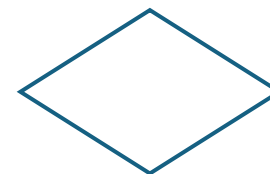
Processo



Entrada de dados



Exibição de dados



Decisão



Conector

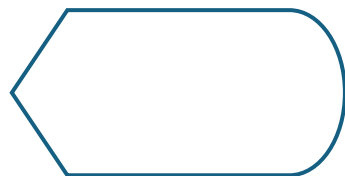
Comandos usados no código:



Entrada de dados



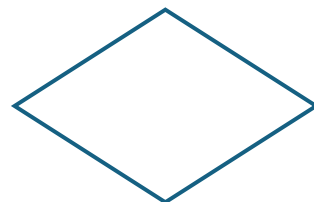
```
X = int(input("Digite X: "))  
Y = float(input("Digite Y: "))
```



Exibição de dados



```
print("O resultado é: %i", %res)  
print("O resultado é: %.2f,%total)
```

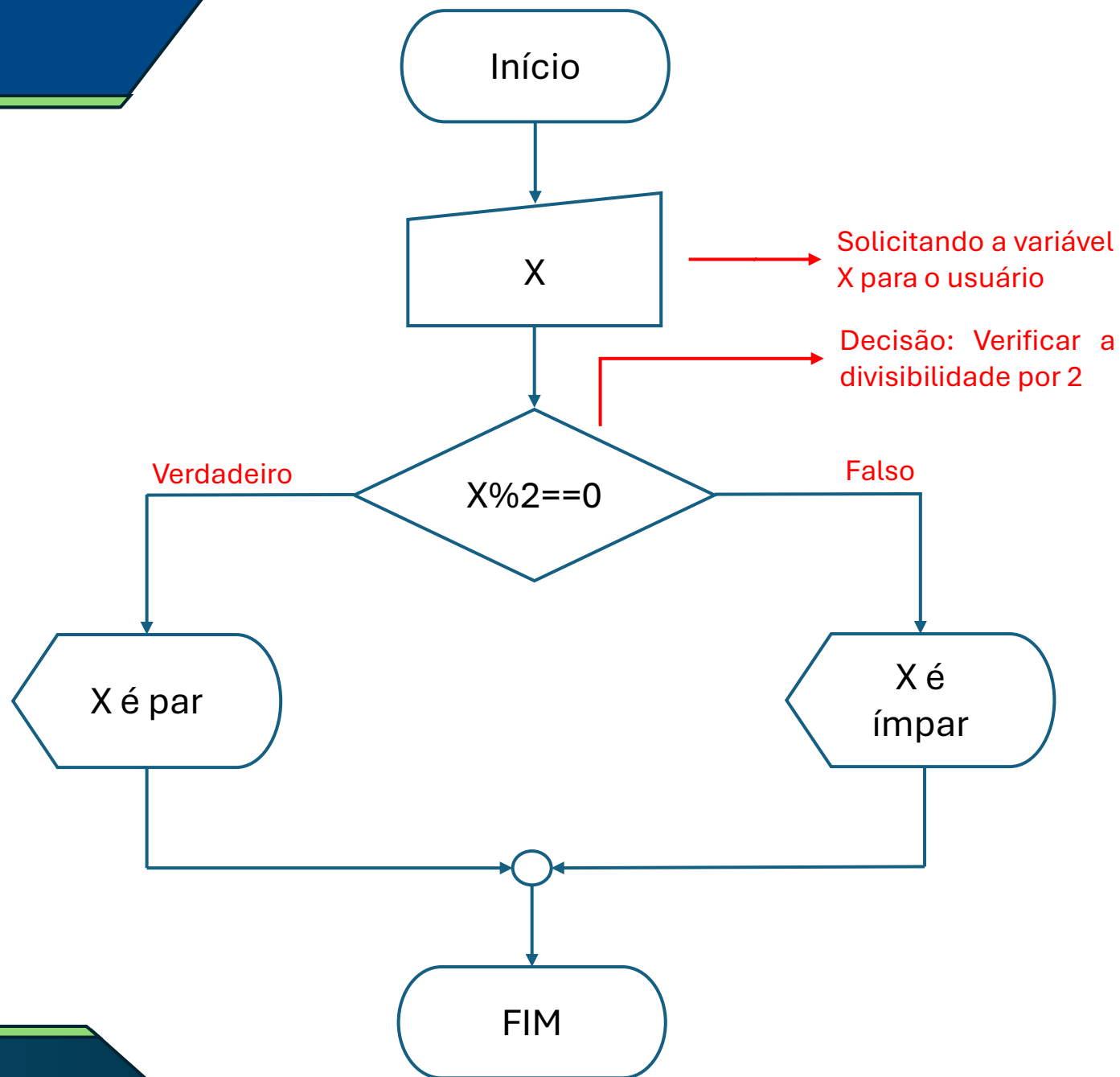


Decisão



```
if x == 5:  
elif y <= 14:
```

Ex: Construa um fluxograma representando um algoritmo onde o usuário digita um número inteiro e o código verifica se o mesmo é par ou ímpar:



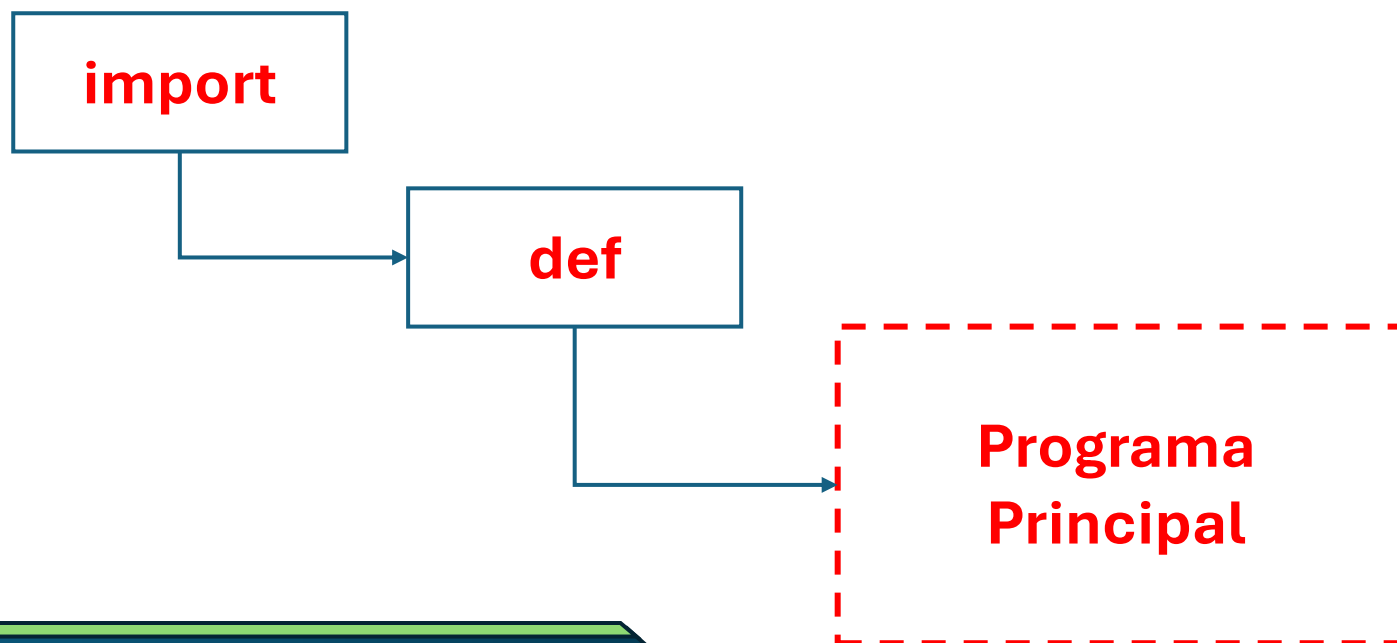
Ex: Converta o fluxograma para um código:

```
X = int(input("Digite X: "))

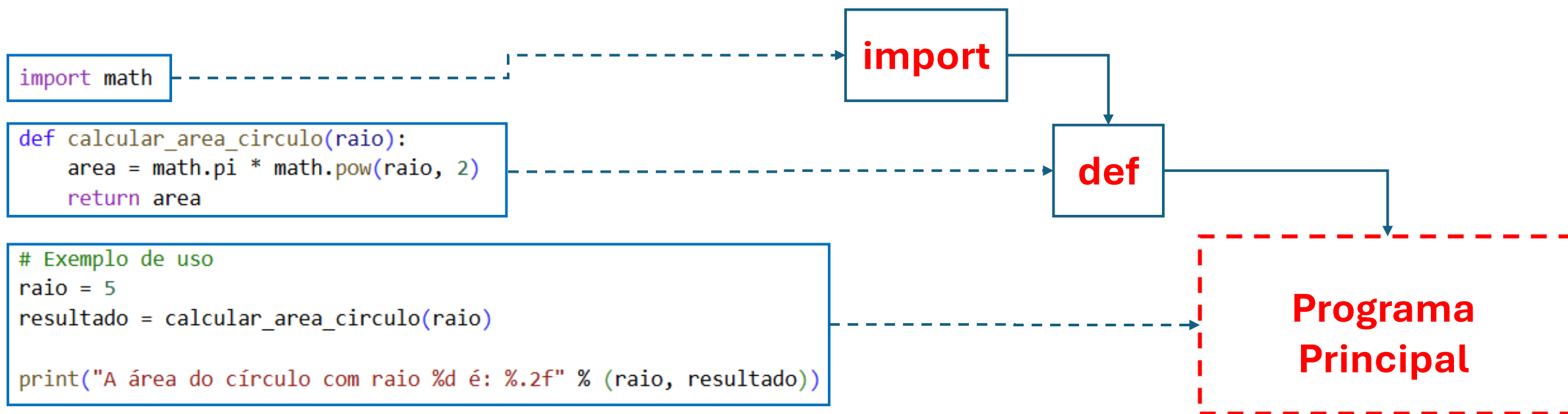
if X%2==0:
    print("O número é par.")
else:
    print("O número é ímpar.")
```

Bibliotecas: Repositório de processos já pré-programados. Normalmente as bibliotecas possuem uma temática (cálculos, gráficos, animações, sistema, etc)

Sub-rotinas: Processos pré-programados pelo próprio usuário para serem chamados em outras sub-rotinas e/ou no programa principal.



Ex: O usuário entra com o raio e o programa calcula a área do círculo através de uma função:



math: A biblioteca **math** é um módulo padrão do Python que fornece funções matemáticas prontas para cálculos mais avançados.



```
import math
```

Para utilizar qualquer comando do **math**, basta colocar o nome da biblioteca, seguido de ponto “.” e depois o nome do comando. Exs:

math.pi -> Retorna o valor de Pi com muitas casas decimais (3.14159...)

math.e -> Retorna o número de Euler (2.71828...)

math.sqrt (variável) -> Retorna a raiz quadrada da variável

math.pow (variável_1 , variável_2) -> Calcula a variável_1 elevada a variável_2

Funções Trigonométricas:

math.sin (variável) -> Calcula o seno da variável

math.cos (variável) -> Calcula o cosseno da variável

math.tan (variável) -> Calcula a tangente da variável

math.radians (variável) -> Converte Graus para Radianos

math.degrees (variável) -> Converte Radianos para Graus

math.exp (variável) -> Calcula $e^{\text{variável}}$

math.log (variável) -> Calcula $\ln(\text{variável})$

math.log10 (variável) -> Calcula $\log_{10} \text{variável}$

math.log2 (variável) -> Calcula $\log_2 \text{variável}$

math.factorial (variável) -> Calcula o fatorial da variável

Arrays: Conjunto de valores numéricos que é utilizado junto com a biblioteca **numpy**. Ao se realizar qualquer cálculo com uma variável **array**, ela realiza um *BROADCASTING* (Transmissão), para todos os valores dentro da estrutura:



```
import numpy as np
```

```
x = np.array([0,3,5,-2,-4,3,8,-1])
```

Cria-se o array

```
y = 2*x
```

Realiza-se uma operação com o array x resultando no array y

```
print(y)
```

```
... [ 0  6 10 -4 -8  6 16 -2]
```

O resultado são todos os elementos do array multiplicados por 2 neste caso.

NumPy significa: **N**umerical **P**ython. É uma biblioteca criada para trabalhar com **cálculos numéricos e científicos** em Python. O principal recurso do NumPy é o **array numérico**, tornando-se vantajosa em relação a biblioteca **math** quando são necessárias cálculos para vários valores.



```
# _____  
#Subrotinas  
import numpy as np
```

Abrevia-se para np

Assim, qualquer comando da biblioteca numpy será com o prefixo **np**:

np.pi -> Retorna o valor de Pi com muitas casas decimais (3.14159...)

np.sqrt (array) -> Retorna a raiz quadrada do array

np.sin (array) -> Calcula o seno do array

np.cos (array) -> Calcula o cosseno do array

np.tan (array) -> Calcula a tangente do array

np.radians (array) -> Converte Graus para Radianos

Formas de se criar arrays:

- Inserção direta:

```
import numpy as np
```

```
x = np.array([0,3,5,-2,-4,3,8,-1])
```

- Passo fixo: **np.arange(a,b,n)** -> Cria um array cujo ponto inicial é “a” e tem como limite o valor “b”. A subdivisão é dada pelo passo “n”

```
import numpy as np

x = np.arange(0,10,1)

print(x)
```

```
... [0 1 2 3 4 5 6 7 8 9]
```

Formas de se criar arrays:

- Número fixo de pontos: **np.linspace(a,b,n)** -> Cria um array com início no ponto “a” e término no ponto “b”. “n” será a quantidade de pontos

```
▶ import numpy as np

x = np.linspace(0,10,10)
y = np.linspace(0,10,11)

print("x=",x)
print("y=",y)
```

```
... x= [ 0.          1.11111111  2.22222222  3.33333333  4.44444444  5.55555556
      6.66666667  7.77777778  8.88888889 10.          ]
    y= [ 0.  1.  2.  3.  4.  5.  6.  7.  8.  9. 10.]
```

Formas de se criar arrays:

np.zeros (n) -> Cria um array com n valores (todos iguais a zero)

np.ones (n) -> Cria um array com n valores (todos iguais a um)

```
▶ import numpy as np

x = np.zeros(5)
y = np.ones(7)

print(x)
print(y)

... [0. 0. 0. 0. 0.]
     [1. 1. 1. 1. 1. 1. 1.]
```


Matplotlib: Biblioteca do Python utilizada para **visualização de dados**. Ela permite criar gráficos de forma simples e rápida.

```
import matplotlib.pyplot as plt
```

→ “Apelido” conhecido

O **Matplotlib** possui uma grande variedade de tipos de gráficos, onde no curso trabalharemos dois deles:

- Dispersão (pontos): **plt.scatter()**
- Linhas: **plt.plot()**

Gráfico de dispersão: Também conhecido como gráfico de pontos, utiliza um plano cartesiano (eixos x e y) e através de pares ordenados $[(x_1, y_1), (x_2, y_2), \dots]$, posiciona-se esses pontos no gráfico:

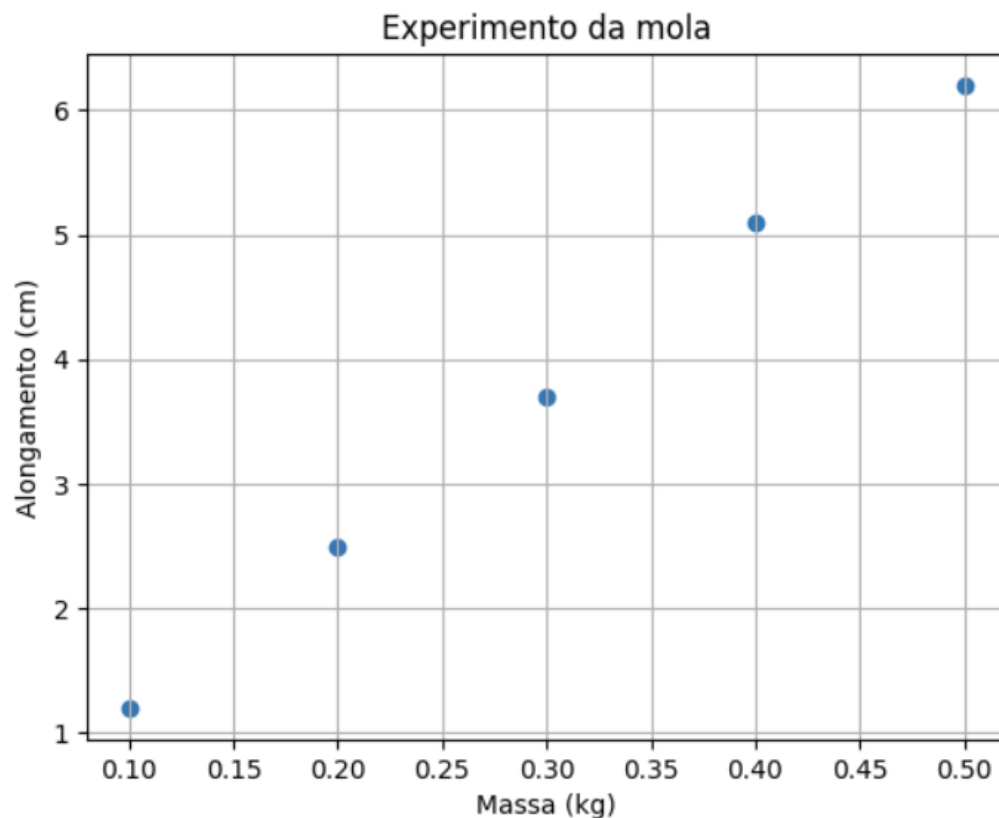
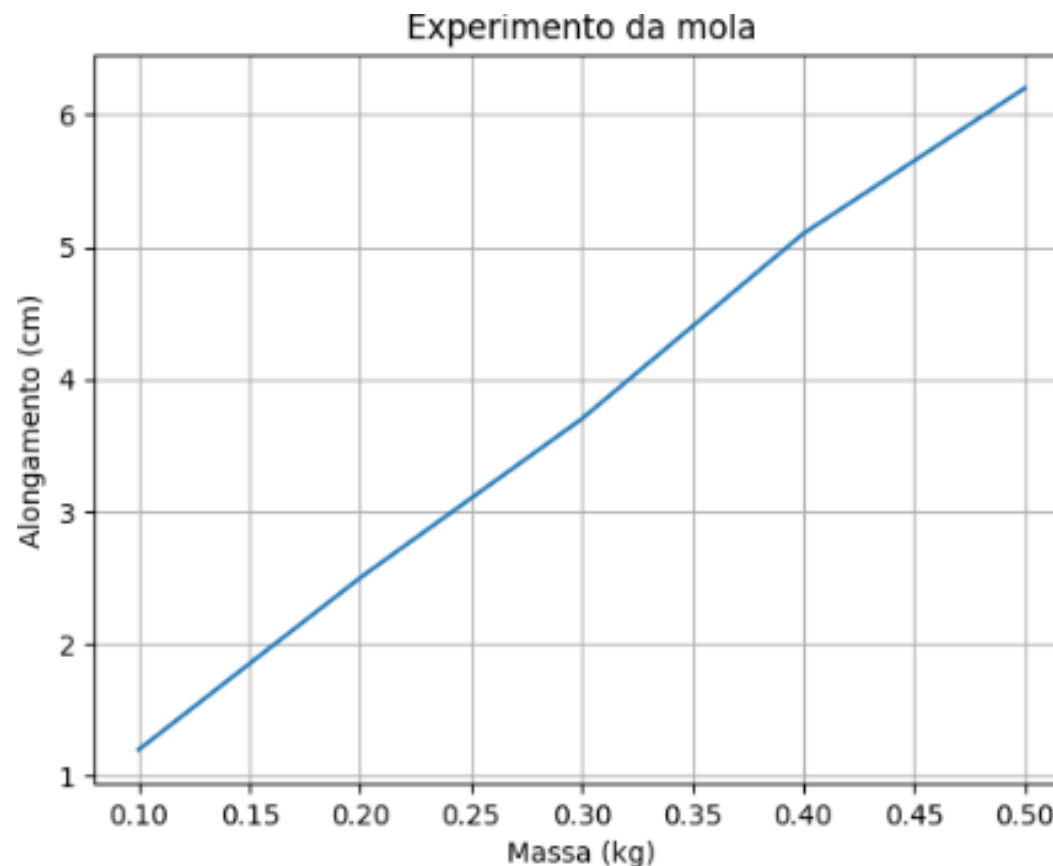


Gráfico de linha: Utiliza um plano cartesiano (eixos x e y) e através de pares ordenados $[(x_1, y_1), (x_2, y_2), \dots]$, liga os pontos através de segmentos de reta.



Configurações gerais dos gráficos:

- **plt.grid()** : Coloca linhas de grade
- **plt.xlabel()** e **plt.ylabel()** : Títulos nos eixos
- **plt.title()** : Título do gráfico
- **plt.legend()**: Legenda dos dados (deve-se utilizar o “label =” no plt.scatter ou plt.plot())
- **plt.show()**: Comando para mostrar o código na tela

Ajustes dentro do comando plt.scatter():

- Cor dos pontos:**

```
import numpy as np
import matplotlib.pyplot as plt

massa = np.linspace(0.1,0.5,5)
alongamento = np.array([1.2,2.5,3.7,5.1,6.2])

plt.scatter(massa, alongamento, color='red')
plt.xlabel("Massa (kg)")
plt.ylabel("Alongamento (cm)")
plt.title("Experimento da mola")
plt.grid()
plt.show()
```

Cor	Código
Azul	'blue'
Vermelho	'red'
Verde	'green'
Preto	'black'
Branco	'white'
Amarelo	'yellow'
Laranja	'orange'
Roxo	'purple'
Rosa	'pink'
Marrom	'brown'
Cinza	'gray'

Ajustes dentro do comando plt.scatter():

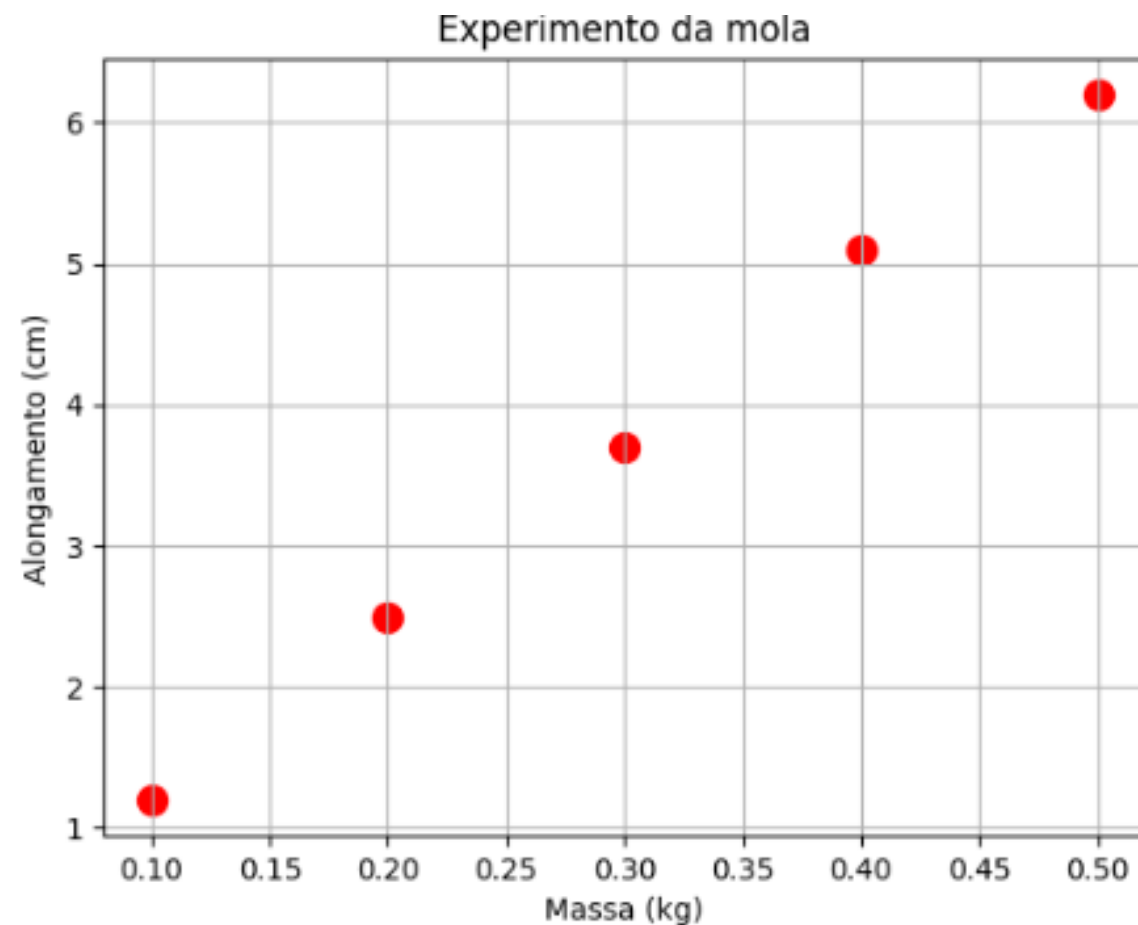
- Tamanho dos pontos:**

```
import numpy as np
import matplotlib.pyplot as plt

massa = np.linspace(0.1,0.5,5)
alongamento = np.array([1.2,2.5,3.7,5.1,6.2])

plt.scatter(massa, alongamento, color='red', s=100)

plt.xlabel("Massa (kg)")
plt.ylabel("Alongamento (cm)")
plt.title("Experimento da mola")
plt.grid()
plt.show()
```



Ajustes dentro do comando plt.scatter():

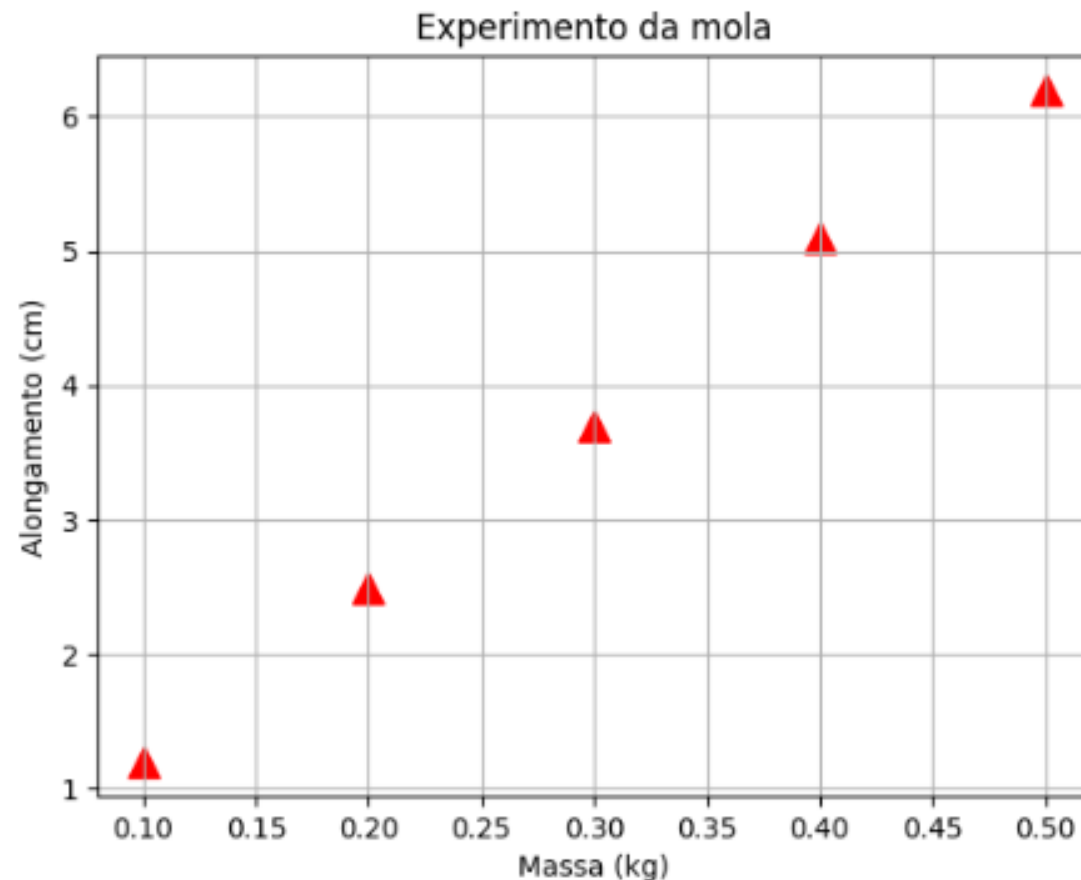
- Formato dos pontos:**

```
import numpy as np
import matplotlib.pyplot as plt

massa = np.linspace(0.1,0.5,5)
alongamento = np.array([1.2,2.5,3.7,5.1,6.2])

plt.scatter(massa, alongamento,color='red',s=120,marker='^')

plt.xlabel("Massa (kg)")
plt.ylabel("Alongamento (cm)")
plt.title("Experimento da mola")
plt.grid()
plt.show()
```



Ajustes dentro do comando `plt.scatter()`:

- **Formato dos pontos:**

```
import numpy as np
import matplotlib.pyplot as plt

massa = np.linspace(0.1,0.5,5)
alongamento = np.array([1.2,2.5,3.7,5.1,6.2])

plt.scatter(massa, alongamento,color='red',s=120,marker='^')

plt.xlabel("Massa (kg)")
plt.ylabel("Alongamento (cm)")
plt.title("Experimento da mola")
plt.grid()
plt.show()
```

Marker	Forma
'o'	círculo
's'	quadrado
'^'	triângulo para cima
'v'	triângulo para baixo
'<'	triângulo esquerda
'>'	triângulo direita
'D'	losango
'p'	pentágono
'*'	estrela
'+'	mais
'x'	X
'.'	ponto pequeno

Ajustes dentro do comando plt.plot():

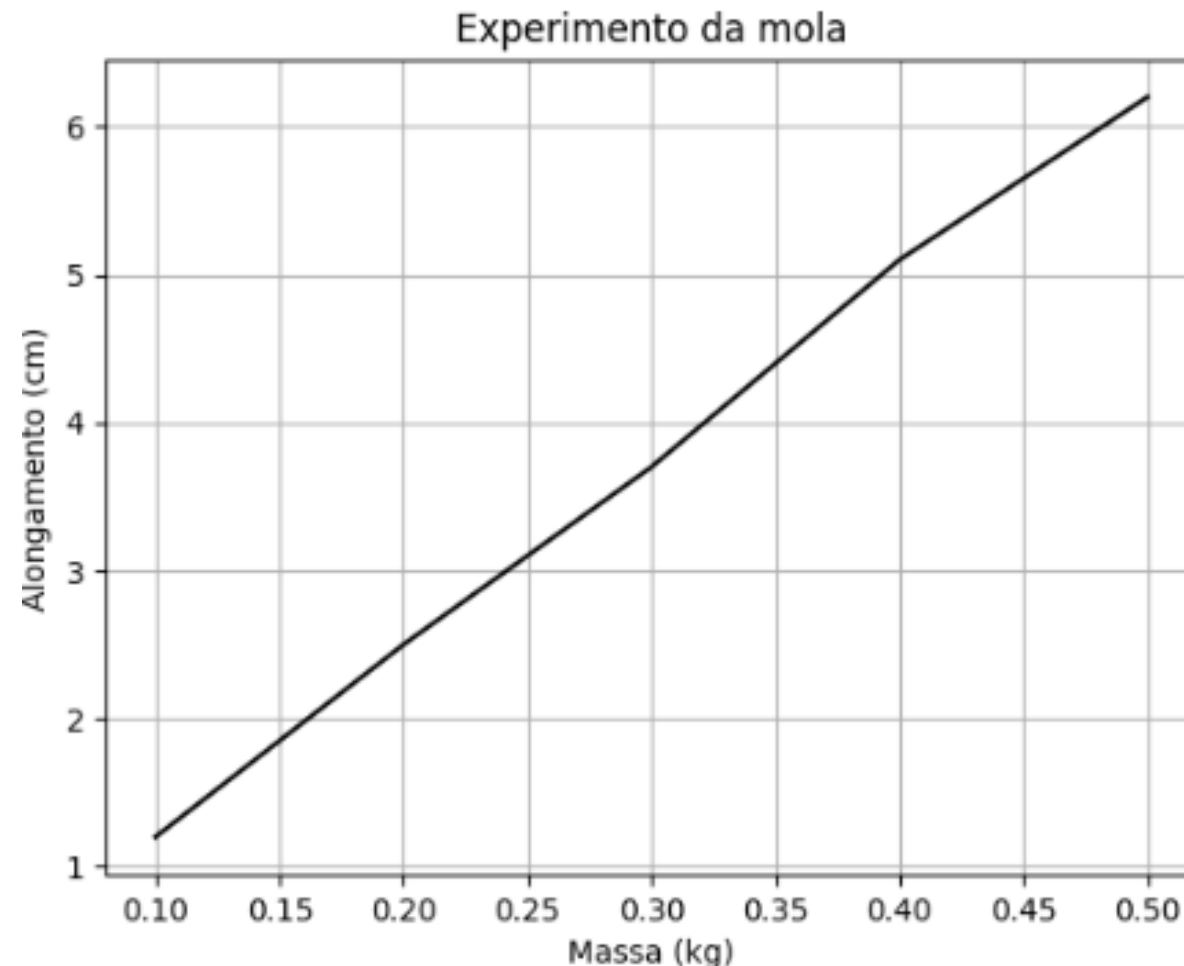
- **Cor da linha:** Mesmo comando do plt.scatter()

```
import numpy as np
import matplotlib.pyplot as plt

massa = np.linspace(0.1,0.5,5)
alongamento = np.array([1.2,2.5,3.7,5.1,6.2])

plt.plot(massa, alongamento, color = 'black')

plt.xlabel("Massa (kg)")
plt.ylabel("Alongamento (cm)")
plt.title("Experimento da mola")
plt.grid()
plt.show()
```



Ajustes dentro do comando plt.plot():

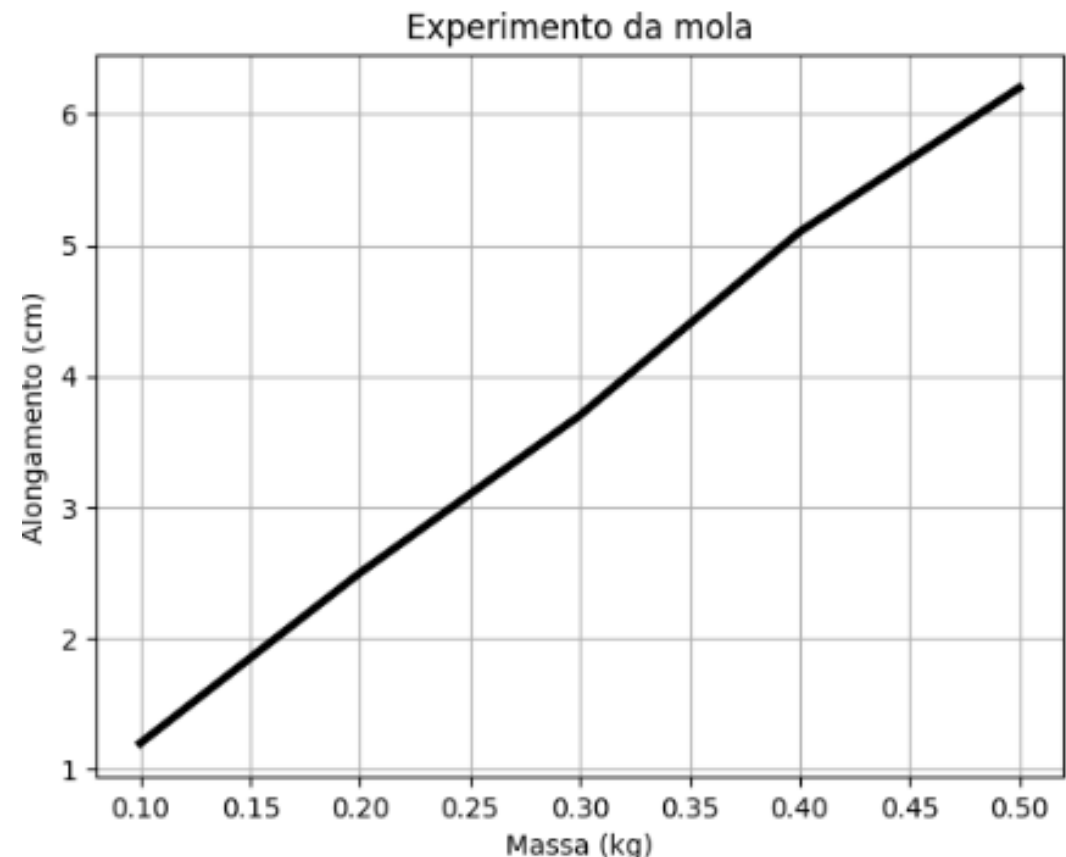
- **Espessura da linha:** Como “linewidth” ou apenas “lw”

```
import numpy as np
import matplotlib.pyplot as plt

massa = np.linspace(0.1,0.5,5)
alongamento = np.array([1.2,2.5,3.7,5.1,6.2])

plt.plot(massa, alongamento, color='black', lw=3)

plt.xlabel("Massa (kg)")
plt.ylabel("Alongamento (cm)")
plt.title("Experimento da mola")
plt.grid()
plt.show()
```



Ajustes dentro do comando `plt.plot()`:

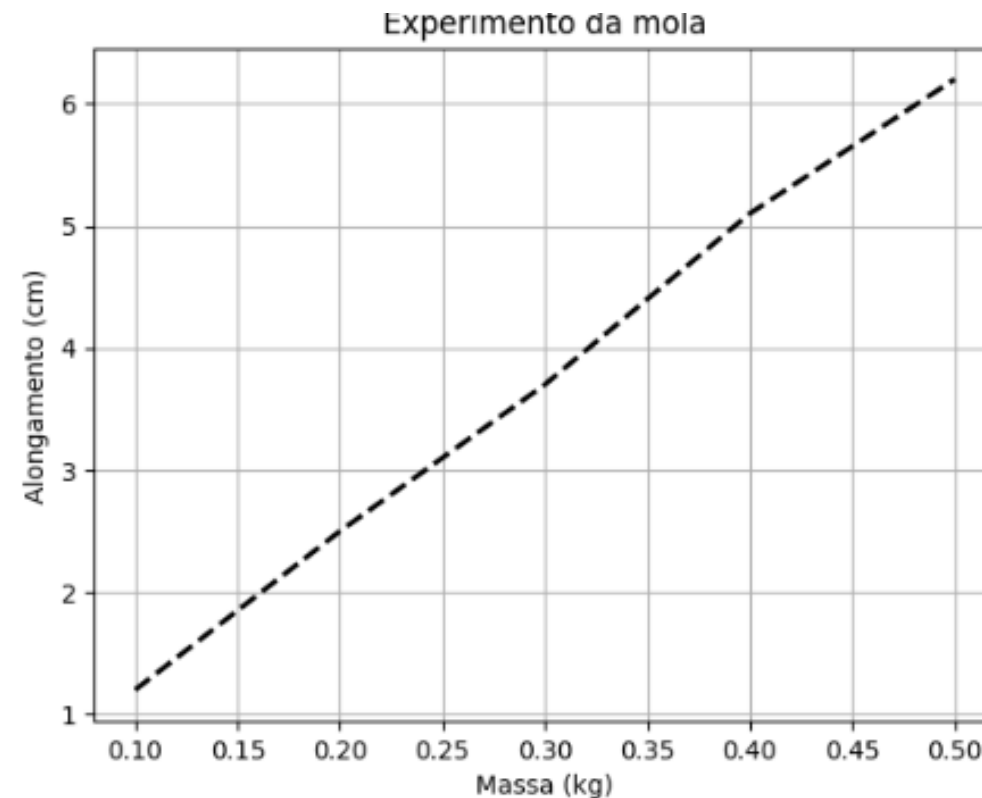
- **Estilo da linha:** Comando “`linestyle`” ou “`ls`”

```
import numpy as np
import matplotlib.pyplot as plt

massa = np.linspace(0.1,0.5,5)
alongamento = np.array([1.2,2.5,3.7,5.1,6.2])

plt.plot(massa, alongamento,color='black',lw=2,ls='--')

plt.xlabel("Massa (kg)")
plt.ylabel("Alongamento (cm)")
plt.title("Experimento da mola")
plt.grid()
plt.show()
```



Código

' - '

' - - '

' - . '

' : '

Tipo de linha

linha contínua

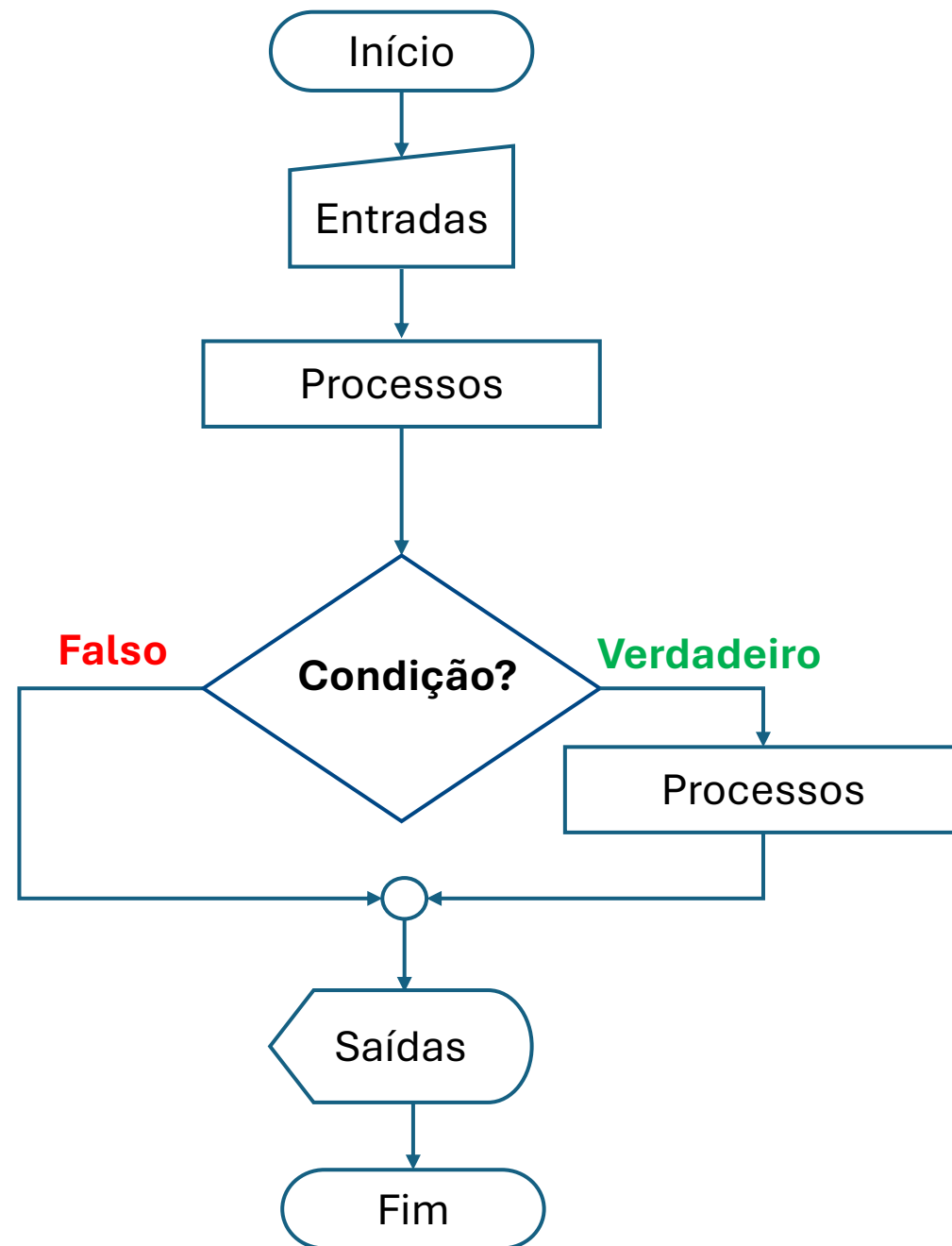
linha tracejada

traço-ponto

pontilhada

Estrutura condicional: Permitirá que o fluxo de informações do código possa sofrer ramificações no seu caminho, dependendo de alguma (ou algumas condições) ocorrer (ocorrerem).

A forma mais simples são duas ramificações: uma com processo e outra sem processo



Variável Booleana: Tipo de variável que pode assumir apenas os valores **VERDADEIRO** e **FALSO**. Proveniente de interações lógicas de decisão ou operações lógicas entre outras variáveis

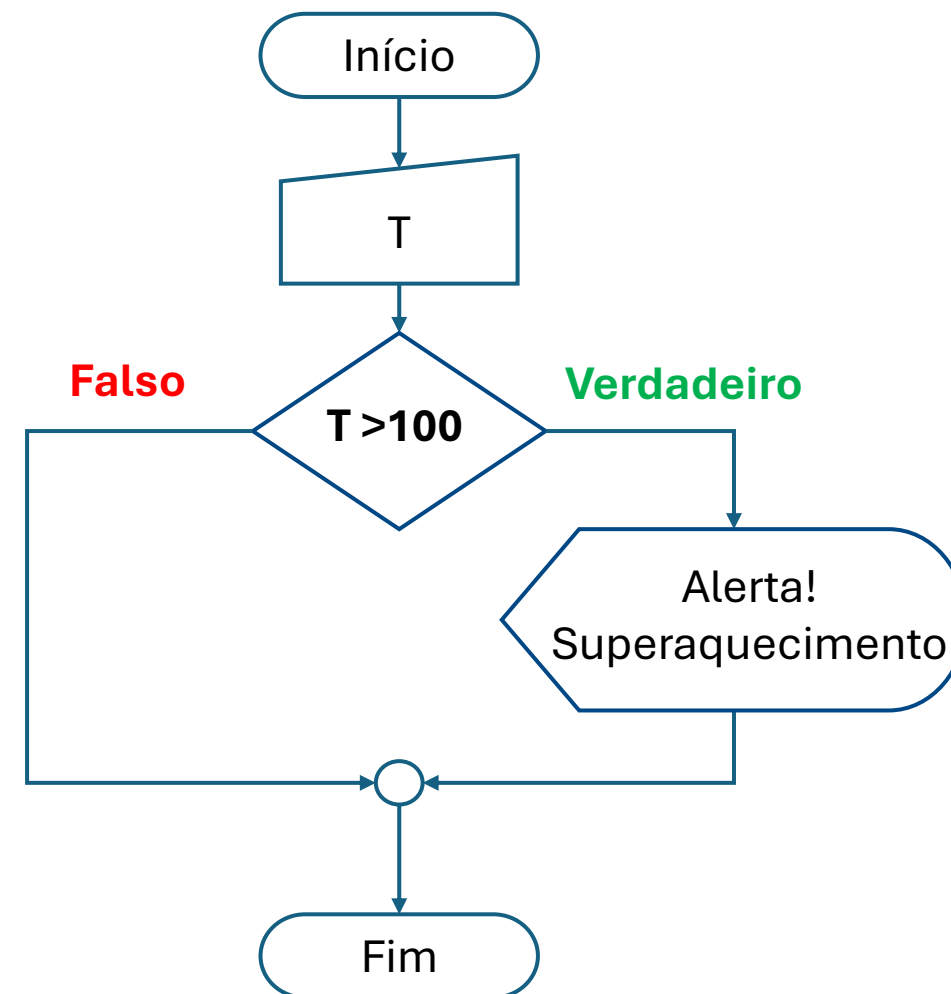
Operador	Significado
==	Igualdade
<	Menor
>	Maior
<=	Menor ou Igual
>=	Maior ou Igual
!=	Diferente

Dois sinais de igualdade
para diferenciar de
processos

Estrutura **“if” simples**: Ramificação em dois caminhos: Verdadeiro (com comandos) e falso (sem comandos)

```
T = float(input("Digite a temperatura atual do motor: "))
```

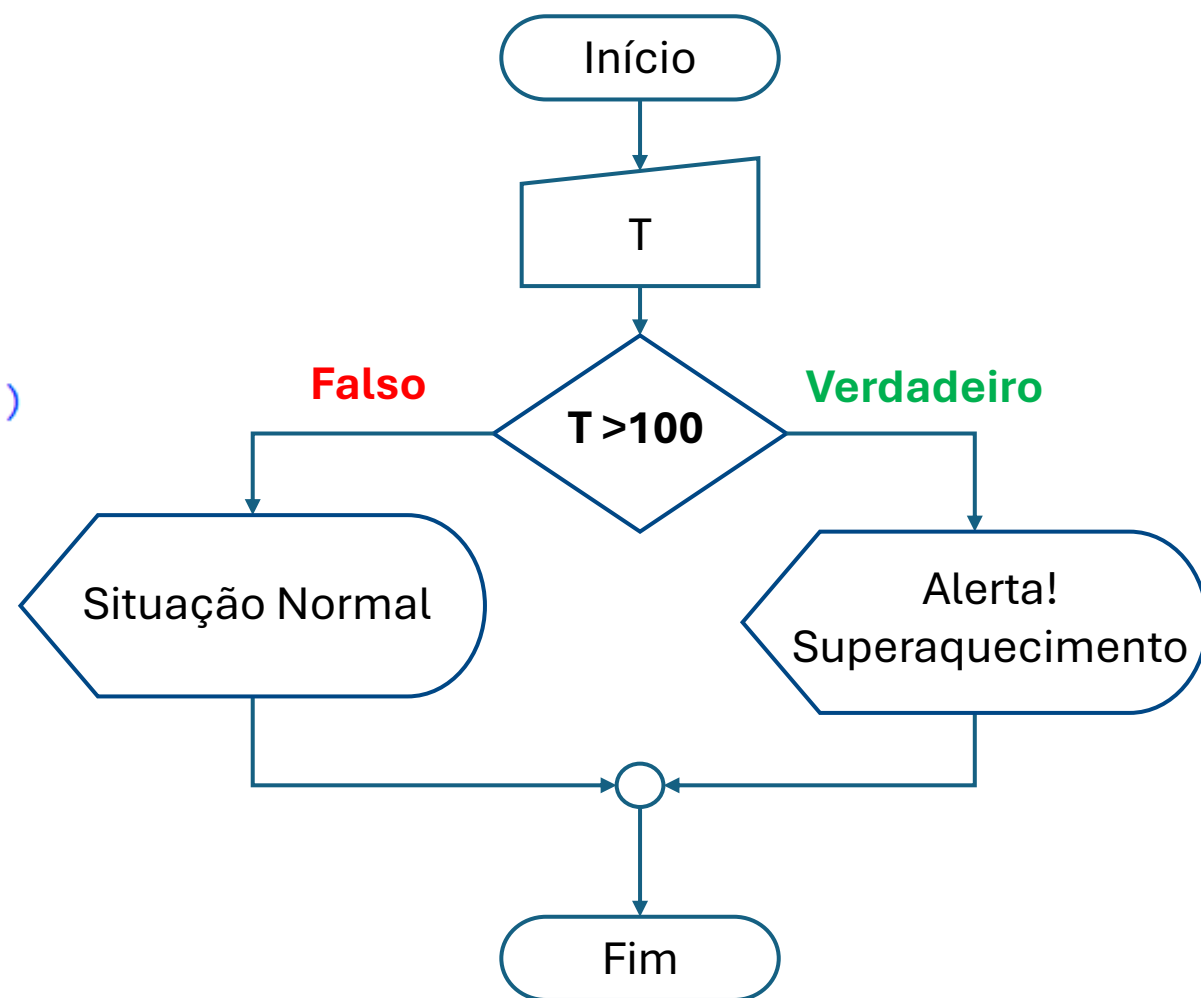
```
if T > 100:
    print("Alerta! Superaquecimento!")
```



Estrutura **“if - else”**: Ramificação em dois caminhos: Verdadeiro (com comandos) e falso (com comandos)

▶ `T = float(input("Digite a temperatura atual do motor: "))`

```
if T > 100:
    print("Alerta! Superaquecimento!")
else:
    print("Situação normal")
```



Estrutura “**if – elif – else**”: Ramificação em três ou mais caminhos:

```
T = float(input("Digite a temperatura atual do motor: "))
```

```
if T > 100:
    print("Alerta! Superaquecimento!")
elif 90 <= T <= 100:
    print("Chegando na região de Superaquecimento")
else:
    print("Situação normal")
```

